

Precious Jackson

CS 499

9/22/2026

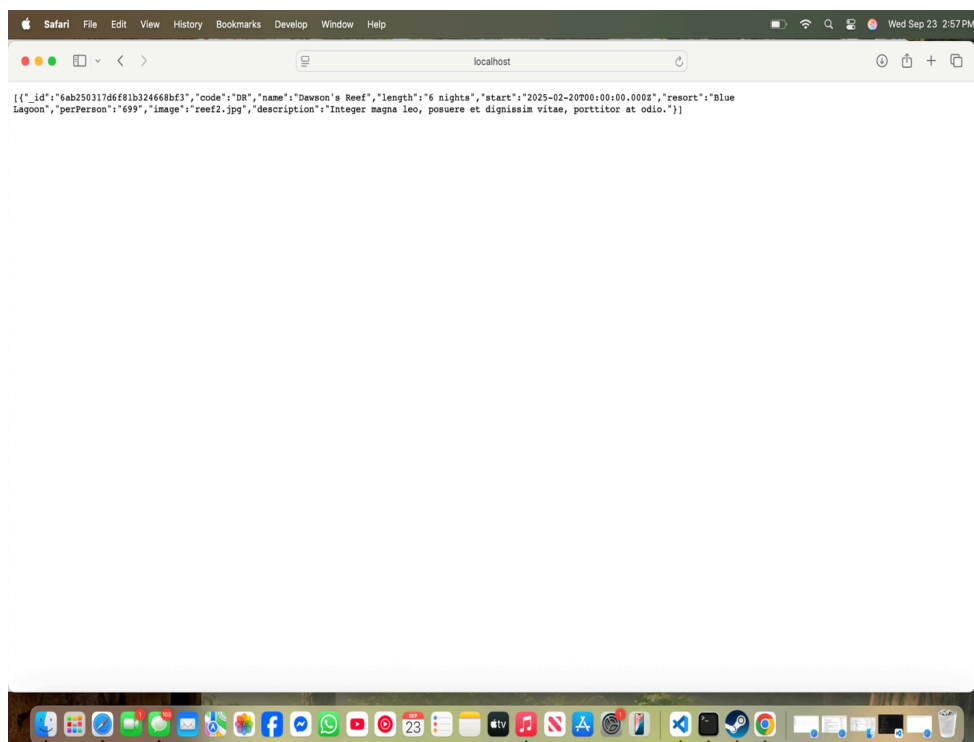
Enhancement One: Software Design and Engineering

The artifact I selected for the Software Design and Engineering category is the Travlr Getaways full-stack web application, which I originally developed in CS 465. The application was designed to provide travel information through a customer-facing website while using an Express server, REST API, and MongoDB database to store and retrieve trip information. I selected this artifact because its full-stack structure gave me an opportunity to demonstrate my growth in software development, application design, security, testing, and maintainability. The original project demonstrated my ability to connect multiple components of a full-stack application, but there were opportunities to improve the way the application processed requests and retrieved information.

For my CS 499 enhancement, I expanded the functionality and design of the Travlr Getaways API. The original trip retrieval functionality was relatively basic because requests primarily returned the complete collection of trip records. I enhanced the application so that the API can process query parameters and dynamically search, filter, and sort trip information. Instead of requiring every request to retrieve the same complete collection, the enhanced application can determine which records should be returned based on the information supplied in the request. These changes make the API more flexible and demonstrate my ability to improve an existing software system without replacing its underlying architecture.

One of the primary software improvements was the addition of search functionality. A user can provide a search term through an API request, and the application evaluates that term when determining which trip records should be returned. For example, during testing I searched for *Dawson*, and the API successfully returned Dawson's Reef instead of the complete trip collection.

Figure 1. API search functionality returning the matching Dawson's Reef record.

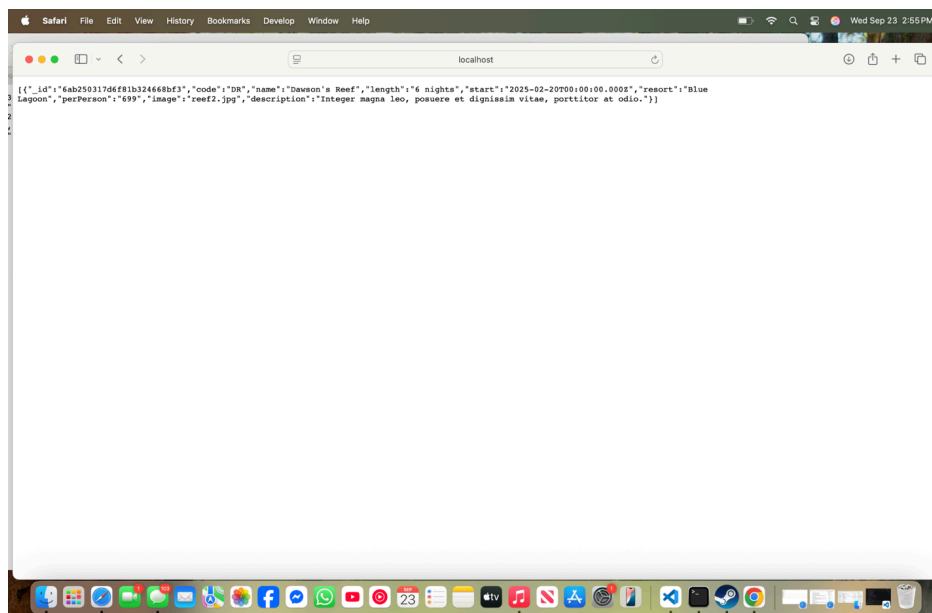


This improvement provides users with a more efficient method of locating relevant information and demonstrates my ability to develop application logic that responds dynamically to user input.

I also implemented filtering functionality to provide additional control over data retrieval.

Filtering allows the application to narrow trip records according to specific characteristics, such as the resort associated with a trip. Rather than creating separate endpoints for every possible category of trip information, query parameters allow the existing endpoint to support different requests.

Figure 2. API filtering functionality returning the matching Blue Lagoon trip record.



This improves the application's design because the API remains reusable while providing more functionality. Search and filtering can also operate together, allowing multiple request conditions to be processed without requiring duplicate retrieval logic.

Another enhancement involved adding sorting functionality. The API can organize the resulting trip records according to an approved field rather than relying on the database's default order.

During testing, sorting by name returned Claire's Reef, Dawson's Reef, and Gale Reef alphabetically.

Figure 3. API sorting functionality returning trip records alphabetically by name.

I limited sorting to approved fields so that the application maintains greater control over the values used to construct the sorting operation. This design decision improves predictability and helps prevent unsupported query values from being used by the application.

These enhancements demonstrate software design and engineering because they required more than simply adding individual features. I had to consider how the new functionality would interact with the existing Express server, API routes, MongoDB database, and customer-facing components of Travlr Getaways. I maintained the existing structure of the application while extending its capabilities through reusable query-processing logic. This approach reduced the need for unnecessary duplicate endpoints and allowed the existing API to support a wider range of requests.

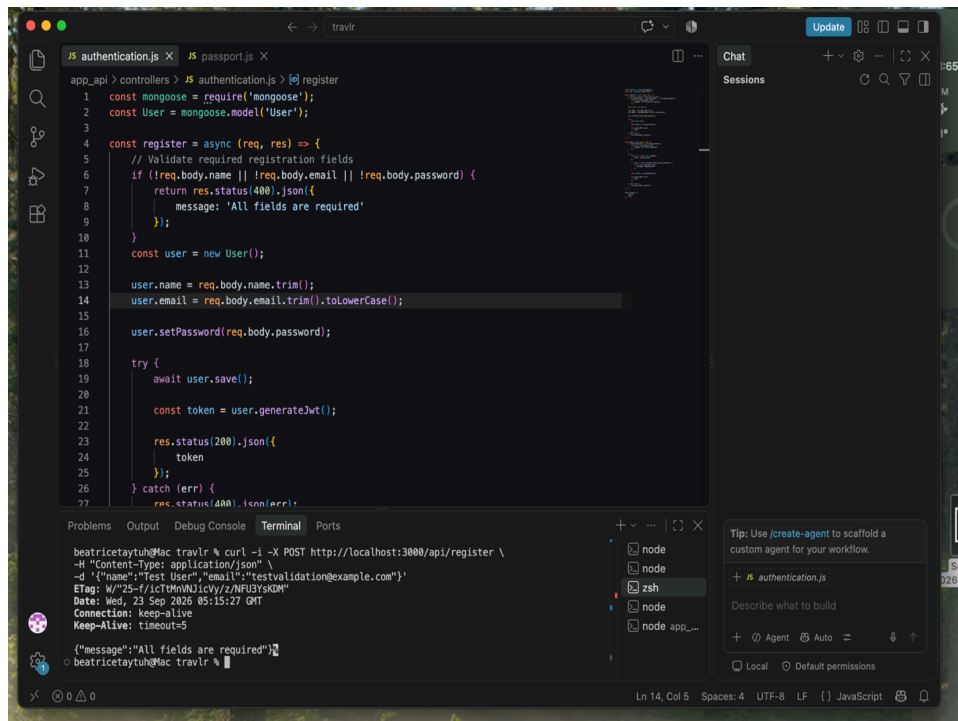
The enhancement also required me to consider maintainability and scalability. Returning every database record for every request may be manageable with a small dataset, but this approach becomes less practical as the number of records increases. Applying search, filtering, and sorting as part of the database retrieval process reduces the amount of irrelevant information returned to the application. Although the current Travlr Getaways dataset is relatively small, designing the application this way creates a better foundation for future expansion. This demonstrates my

ability to consider how a software design decision can affect the application beyond its immediate requirements.

Security was another important consideration while working with the application. Travlr Getaways uses Passport and JSON Web Token authentication as part of its existing authentication architecture. During implementation and testing, the application required a JWT secret for the authentication strategy to operate correctly. I configured the application to load this value through an environment variable rather than placing the authentication secret directly into the application's primary source code. Separating sensitive configuration information from the main application logic is an important design practice because it improves maintainability and reduces unnecessary exposure of configuration values.

I also strengthened the application's handling of user-supplied query parameters by escaping special regular-expression characters before search and resort values are used to construct MongoDB queries. This defensive-programming improvement helps prevent client input from unintentionally changing the behavior of the application's search logic. Together with the existing authentication configuration and input validation, this change demonstrates my consideration of security throughout the application's design rather than treating security as a separate feature.

Figure 4. Registration input validation rejecting a request with missing required fields.



Working with the authentication configuration also helped me understand the relationship between security and overall software design. When the required JWT configuration was unavailable, the application could not initialize its authentication strategy correctly. Resolving this issue required me to trace the problem through the application's configuration and verify that the environment variable was available when the server started. This troubleshooting process demonstrated that security components cannot be treated as isolated additions; they must be correctly integrated with the rest of the application's architecture.

Testing was an important part of evaluating whether my design changes worked as intended.

Figure 5. Travlr Getaways trip management interface displaying existing trip records.

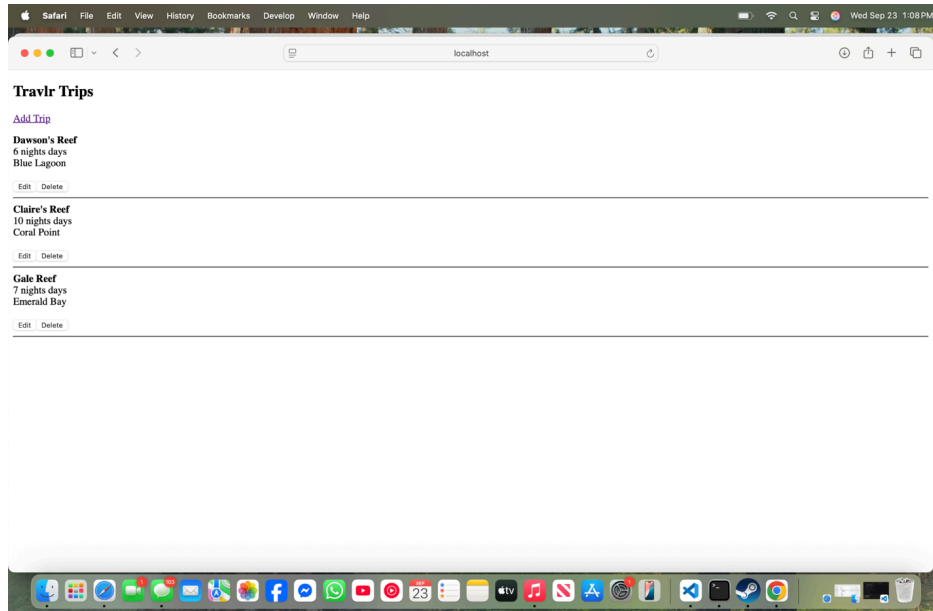


Figure 6. Successful creation of a new trip record through the enhanced application.

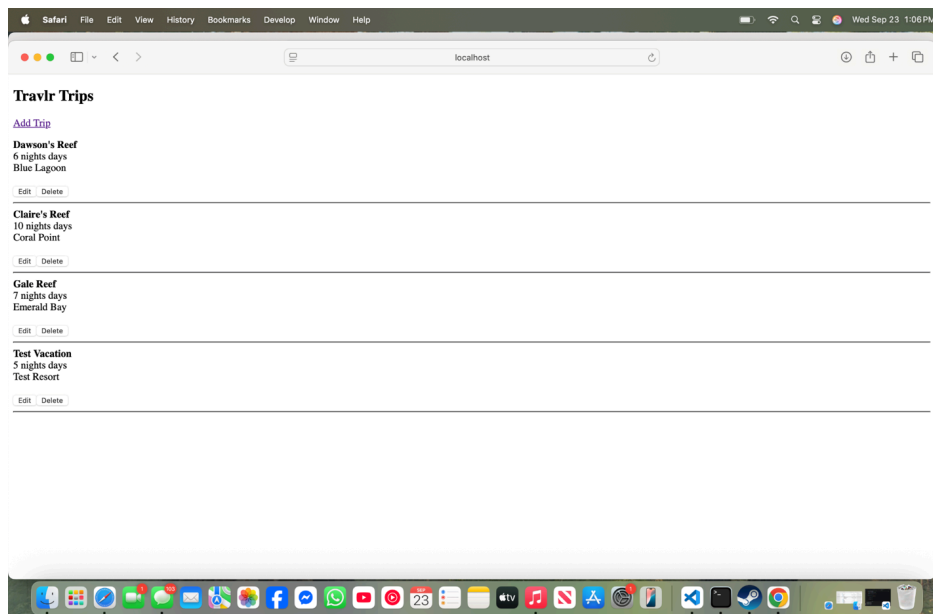
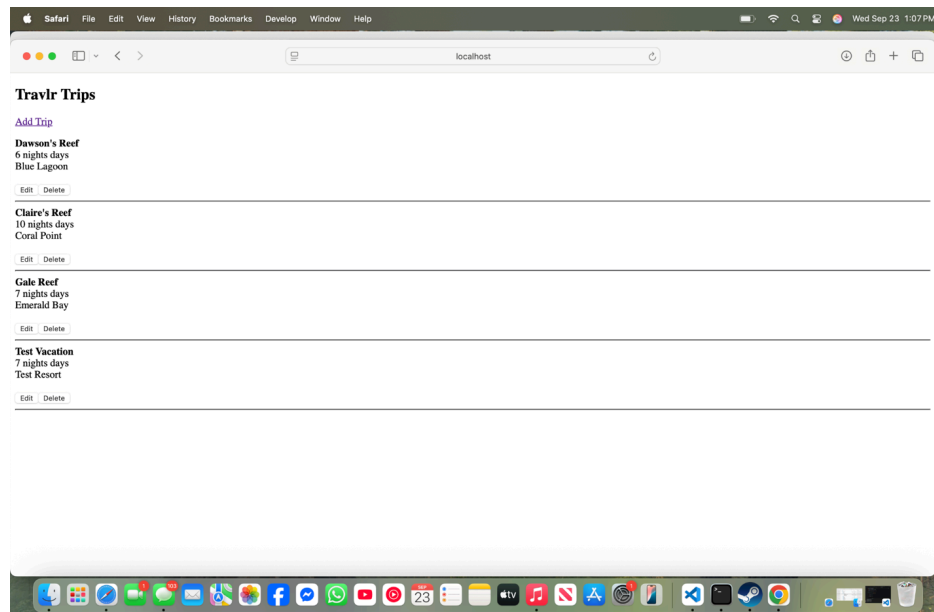


Figure 7. Successful update of an existing trip record from five to seven nights.



I first verified that the original `/api/trips` endpoint continued to return the complete trip collection. This was important because an enhancement should not unnecessarily break existing functionality. I then tested the search functionality and confirmed that searching for Dawson returned Dawson's Reef. I separately tested sorting and confirmed that records could be returned alphabetically by name. Finally, I tested combinations of query parameters to determine whether the application could process multiple criteria while maintaining the existing API functionality.

The testing process also helped me identify and resolve problems outside of the new retrieval functionality. During implementation, I encountered configuration and application startup issues involving authentication, environment variables, server execution, and the application's port. Resolving these problems required me to examine how different components of the application interacted rather than treating each error independently. After correcting the configuration issues,

I verified that the Express application started successfully, MongoDB connected, and the API returned the expected trip information.

This enhancement demonstrates several skills associated with the Software Design and Engineering category. First, I demonstrated the ability to develop a design solution that improves an existing application instead of rebuilding it unnecessarily. Second, I programmed logic that allows the API to solve different retrieval problems depending on the parameters supplied by the user. Third, I considered security by working with the application's authentication configuration and separating the JWT secret from the main application source code. Finally, I tested the working product to verify that both the existing and enhanced functionality operated correctly.

The enhancement also demonstrates progress toward the CS 499 course outcomes because it required me to evaluate an existing computing artifact, identify opportunities for improvement, implement those changes, and communicate the reasoning behind my design decisions. Rather than adding functionality without considering its effect on the rest of the application, I considered how the API, database, authentication configuration, routing, and server components worked together. This process strengthened my understanding of software engineering as the design and maintenance of an interconnected system.

My original enhancement plan focused on evaluating and improving the architecture, functionality, validation, error handling, security, and maintainability of Travlr Getaways. The completed enhancement remains consistent with that overall goal. The implementation process required some additional troubleshooting that I did not initially anticipate, particularly with the

application's authentication and environment configuration. However, resolving those issues became a valuable part of the enhancement because it strengthened my understanding of configuration management and the dependencies between different parts of a full-stack application.

Overall, enhancing Travlr Getaways allowed me to build upon my original CS 465 artifact and demonstrate the software engineering skills I have developed throughout the computer science program. The application now provides more flexible trip-data retrieval while preserving its existing full-stack architecture. The enhancement demonstrates my ability to analyze an existing application, design and implement new functionality, address security considerations, troubleshoot integration problems, and test the resulting product. These improvements make Travlr Getaways a stronger representation of my current abilities in software design and engineering.